

Fast and Robust Kernel Generators for Star Trackers

David Arnas*, Márcio A. A. Fialho[†] Daniele Mortari[‡]

Abstract

Star Identification (Star-ID) is a complex problem, mainly because some of the observations are not generated by actual stars, but by reflecting debris, other satellites, visible planets, or by electronic noise. For this reason, the capability to discriminate stars from non-stars is an important aspect of Star-ID robustness. Usually, the Star-ID task is performed by first attempting identification on a small group of observed stars (a kernel) and, in case of failure, replacing that kernel with another until a kernel made only of actual stars is found. This work performs a detailed analysis of kernel generator algorithms, suitable for on-board implementation in terms of speed and robustness, for kernels of three (triad) and four (quad) stars. Three new kernel generator algorithms and, in addition to the existing *expected time to discovery*, three new metrics for robustness evaluation are proposed. The proposed algorithms are fast, robust to find good kernels, and do not require pre-stored data.

1 Introduction

Star trackers, also known as star sensors or star cameras, are currently the most accurate attitude sensors, being able to provide absolute attitude measurements with errors smaller than few arc-seconds [1]. For this reason, they are used when precise attitude determination is required [2]. Moreover, the faster the star tracker acquires and updates the attitude measurements, the better the spacecraft attitude control subsystem performs its task.

One of the most critical process of attitude determination using star trackers is the Star IDentification (Star-ID). Many of the most successful Star-ID algorithms select first a small number of observed stars (a kernel)

*Ph.D. Student, Centro Universitario de la Defensa Zaragoza, Crta. Huesca s/n, 50090 Zaragoza, Spain. Email: DARNAS@UNIZAR.ES

[†]Doctorate Student, Divisão de Eletrônica Aeroespacial, INPE - Instituto Nacional de Pesquisas Espaciais, São José dos Campos, SP, Brazil. CEP: 12227-010. E-mail: MARCIO.FIALHO@INPE.BR

[‡]Professor, Aerospace Engineering, Texas A&M University, College Station, TX 77843-3141. E-mail: MORTARI@TAMU.EDU

and attempt the identification of these stars before proceeding with the identification of the remaining observed stars [3, 4, 5, 6]. If one or more stars in a kernel is not a star, the identification fails, and the Star-ID algorithm needs to select another kernel from the same image, or reject the current image and request another one. Hence, the performance of many star trackers depends critically on how well the kernels of observed stars are replaced. One of the first algorithms to generate sequences of kernels different from the trivial “three inner loops” (of the lexicographic sequence) was described in [6]. More recently, a detailed study, performed in [7], found an optimal solution for some cases, providing very good solutions in the remaining situations. However, some of the algorithms proposed in [7] are not feasible to be implemented in a star tracker, due to their heavy computational requirements or large amount of memory used. In fact, an important requirement for star trackers is that the algorithms must run in an embedded low power computer with limited memory and computational power [8, 9]. These limitations become even more severe in high reliability star sensors developed for geostationary and deep space missions [10, 11].

In this paper we propose three new algorithms for generating sequences of kernels. These algorithms generate all possible kernels (combinations) with no repetitions. They are simple, fast, and do not require pre-stored data or any significant amount of memory. In particular, we focus on versions of these algorithms that generate sequences of triads (three stars) or quads (four stars) of kernels. The main reason quads are considered is because, under the hypothesis of uniform distribution of stars, the probability of getting similar quads in the star catalog drops down several orders of magnitude with respect to triads, as shown in [12]. Two-star kernels are not considered because it is practically impossible to get an unambiguous Star-ID within the measurements accuracy. On the other hand, no Star-ID approach has been proposed for kernels of five or more stars. The proposed algorithms fall under the category of “Stateless Sequence Generation Algorithms” described in [7].

This paper is organized as follows: Sections 2, 3, and 4 contain the description of these three new sequence generator algorithms. In Sections 5 and 6, a comparison of these algorithms with older algorithms described in the literature is performed in terms of their robustness and speed, respectively. In addition, three new metrics for evaluating kernel sequence generator algorithms are introduced. These metrics complement the objective function presented in [7].

2 Enhanced Pattern Shifting

2.1 Previous algorithms: Lex and PS

Before introducing the *Enhanced Pattern Shifting* (EPS) algorithm, a brief description of the *Lexicographic* (Lex) and the original *Pattern Shifting* (PS) sequence generators is presented. The original PS algorithm is the base algorithm that lead to the EPS development.

Lex is the simplest kernel generator that can be devised for the task of generating all kernels without repetition. Its triad version consists of just three nested loops:

Algorithm 1: Triad generator using Lexicographic ordering

```

for  $i$  from 1 to  $(n-2)$  do
  for  $j$  from  $(i+1)$  to  $(n-1)$  do
    for  $k$  from  $(j+1)$  to  $n$  do
      | next kernel is “[ $i, j, k$ ]”;
    end
  end
end

```

where n is the number of observed stars, and i , j , and k are the stars indexes. The Lex algorithm is one the worst kernel sequence generators as the index associated with the most external loop persists for many consecutive kernels. This fact motivated the development of the Pattern Shifting (PS) algorithm, introduced with Pyramid [6]. The idea behind PS is to replace star indexes by their differences as loop counters. This creates sequences with much less index repetitions. The PS algorithm has the following pseudo-code for triads:

Algorithm 2: Triad generator using Pattern Shifting

```

for  $dj$  from 1 to  $(n-2)$  do
  for  $dk$  from 1 to  $(n-dj-1)$  do
    for  $i$  from 1 to  $(n-dj-dk)$  do
      |  $j = i + dj$ ;
      |  $k = j + dk$ ;
      | next kernel is “[ $i, j, k$ ]”;
    end
  end
end

```

2.2 The Enhanced Pattern Shifting (EPS) algorithm

By including an additional inner loop to PS, the EPS algorithm is obtained. This additional loop causes all star indexes to increment by the same amount, κ , equal to the size of kernels. If n is sufficiently large, kernels with less index repetitions are obtained. For kernels of triads, the EPS pseudo-code is:

Algorithm 3: Triad generator using Enhanced Pattern Shifting

```

for  $dj$  from 1 to  $(n-2)$  do
  for  $dk$  from 1 to  $(n-dj-1)$  do
    for  $ii$  from 1 to 3 do
      for  $i$  from  $ii$  to  $(n-dj-dk)$  in steps of 3 do
         $j = i + dj$ ;
         $k = j + dk$ ;
        next kernel is “[ $i, j, k$ ]”;
      end
    end
  end
end

```

EPS generates better kernel sequences than PS for $n \geq 2\kappa$, that is, $n \geq 6$ for triads and $n \geq 8$ for quads. For example, for $n = 6$, the EPS algorithm generates: 1-2-3, 4-5-6, 2-3-4, 3-4-5, 1-2-4, $\dots$, whereas PS provides: 1-2-3, 2-3-4, 3-4-5, 4-5-6, 1-2-4, $\dots$. This simple example shows the better variations of EPS kernels. However, for $n < 2\kappa$, the sequences generated by them are identical. A thoroughly and more detailed comparison is performed in Section 5. EPS can easily be generalized for generating kernels of an arbitrary size κ . A version for quads is shown in the Appendix.

3 Shifting Necklaces

Shifting Necklaces (SN)¹ is an algorithm based on the idea of generating a pattern (the necklace) that is shifted a certain amount of steps in the sequence of kernels. Once this pattern has fulfilled all its possible kernels, the algorithm changes the pattern and repeats the process, generating all the possible kernels with no repetition.

3.1 The necklace problem

The necklace problem is a combinatorial problem which answers the question of how many different arrangements of n pearls in a circular loop can be produced, assuming that each pearl comes in one of k different colors. In

¹Pseudo-code listings for this algorithm are provided in the Appendix.

this definition, two arrangements are considered to be identical if they only differ by a rotation inside the loop. The number of different arrangements is given by the application of the Burnside's counting theorem [13], which can be summarized in the following expression:

$$N_k(n) = \frac{1}{n} \sum_{d|n} \varphi(d) k^{n/d}, \quad (1)$$

where the sum is taken over all the divisors d of n , and $\varphi(d)$ is called the Euler's totient function of d , an arithmetic function that counts the number of positive integers less than or equal to d that are coprime with d . The case of study is a simplification of the general necklace problem where $k = 2$, as only two different states for each position are possible, the first one having the position (star) selected, and the second, the case in which it is not.

3.2 Generating the necklace

Let n be the number of observed stars in an image. For the purpose of the necklace problem, n represents the available positions (the observed stars) whilst the necklace corresponds to the actual triad or quad of stars chosen. An example of necklace can be seen in Fig. 1, where 4 different stars are in study. In the figure, 3 stars are selected (the colored circles) forming the necklace.

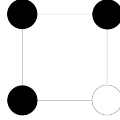


Figure 1: Example of necklace.

Mathematically, a necklace is represented by the subset $\mathcal{G} \subseteq \mathbb{Z}_n = \{1, \dots, n\}$, where $\mathbb{Z}_n$ is a ring of integers. Thus, taking as an example Fig. 1, and ordering the positions from the upper left corner in a clockwise orientation, it leads to $\mathcal{G} = \{1, 2, 4\}$. However, this is not the only representation that correspond to this necklace. In fact, the following necklaces are identical (see Fig. 2):

$$\mathcal{G} = \{1, 2, 4\} = \{1, 2, 3\} = \{2, 3, 4\} = \{1, 3, 4\}, \quad (2)$$

where the difference between them is just a rotation in the circular loop, not changing the distribution in the process. Two necklaces ($\mathcal{G}_1$ and $\mathcal{G}_2$) are considered to be identical if they fulfill the following relation:

$$\mathcal{G}_1 = \mathcal{G}_2 \iff \exists s : \mathcal{G}_1 = (\mathcal{G}_2 + s) \bmod n, \quad (3)$$

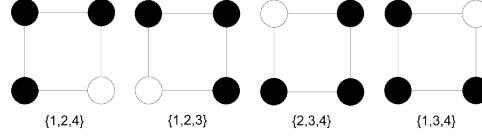


Figure 2: Identical necklaces.

where s is an integer. This is an important property because the objective of the algorithm is to generate a sequence of kernels without any repetitions, so ambiguity problems in the representation have to be addressed.

In order to overcome this problem, each necklace is represented using the distances between the positions in the necklace instead of their real positions. That way, all configurations from Fig. 2 can be represented by their distances, that is, $\delta\mathcal{G} = \{1, 2, 1\}$. Note that using this notation, there exists multiple representations that correspond to the same distribution due to the modular nature of the problem. However, there are some advantages of using this representation. First, it is easier to find equivalent representations by an observation of the values of $\delta\mathcal{G}$, which makes the computations faster. Second, there is an additional relation between the values of $\delta\mathcal{G}$.

Let $\delta\mathcal{G}$ be a distribution in distances, then, all possible representations ($\hat{\delta\mathcal{G}}$) are the set consisting of:

$$\hat{\delta\mathcal{G}} = \bigcup_{s=0}^{n-1} \{\delta\mathcal{G}([1 + s, \dots, i + s, \dots, n + s] \bmod n)\}. \quad (4)$$

All other permutations lead to different necklaces. Moreover, due to the modular nature of the problem, all possible distributions fulfill the following property:

$$\sum_{i=1}^{\kappa} \delta\mathcal{G}(i) = n, \quad (5)$$

where κ is the kernel size (3 for triads, 4 for quads), n is the number of observed stars, and where all values of $\delta\mathcal{G}(i) \in [1, (n - \kappa + 1)]$. This leads to a number of degrees of freedom of $(\kappa - 1)$ when defining each necklace. Therefore, $\delta\mathcal{G}(i)$ with $i = \{1, \dots, (\kappa - 1)\}$ are used as the variables of the problem.

Now, all possible necklaces are required to be generated without any repetition. Equation (4) shows that there are representations that are equivalent, and thus, a criteria has to be established in order to avoid these ambiguities in the representation. Without losing generality, an initial set of necklaces with the following property is considered:

$$\delta\mathcal{G}(1) \leq \delta\mathcal{G}(2) \leq \dots \leq \delta\mathcal{G}(\kappa - 1) \leq \delta\mathcal{G}(\kappa), \quad (6)$$

which defines a subset from the possible necklaces. However, the remaining can be built by permutations of this initial set. From Eqs. (4), (5), (6) we

can derive that this initial subset of necklaces can be defined as:

$$\delta\mathcal{G}(i) \in \left[\delta\mathcal{G}(i-1), \left\lfloor \frac{n - \sum_{j=1}^{i-1} \delta\mathcal{G}(j)}{\kappa - i + 1} \right\rfloor \right], \quad \delta\mathcal{G}(1) \in \left[1, \left\lfloor \frac{n}{\kappa} \right\rfloor \right], \quad \forall i \in [2, \kappa - 1] \quad (7)$$

where $\lfloor x \rfloor$ is the round down of x to the next integer, being x a real number. Note that with this definition, all the values of $\delta\mathcal{G}(i)$ fulfill Eq. (6).

This initial set of necklaces is able to generate all necklace combinations just by permutations in the positions of $\delta\mathcal{G}(i)$. One important thing to notice is that if there are equal values of $\delta\mathcal{G}(i)$, some permutations generate the same necklace, and thus, they have to be rejected in the process.

3.3 Performing the shifting of the necklace

Once a necklace is built, two parameters are required to be defined in order to generate the shifting: the shifting step (d_{step}) and the number of steps that each necklace has to be shifted (n_{step}). Those values are dependent on each particular necklace that is shifted, which increases the computational time but reduces the number of stars that are repeated between steps.

3.3.1 Shifting step

Ideally, the shifting step, d_{step} , is an integer such that there is no simple linear combination of $\delta\mathcal{G}(i)$ equal to d_{step} , that is:

$$\sum_{i=1}^{\kappa} a_i \delta\mathcal{G}(i) \neq d_{step}, \quad (8)$$

where the a_i are integers such that $a_i = \{0, 1\}$. This is a very strong condition that may not always be fulfilled depending on the necklace and the number of observed stars. However, it is important to minimize the number of possible linear combinations of Eq. (8) because they control the number of stars that are repeated between steps.

One thing to notice is that the computation of the shifting step is only required to be done once for the initial subset of necklaces $\delta\mathcal{G}$ generated, not needing further calculations to obtain the shifting step d_{step} when permutations of $\delta\mathcal{G}(i)$ are performed.

3.3.2 Number of steps

The objective now is to find the number of steps that a particular necklace can be shifted without generating repetitions in the sequence. The idea behind this methodology is that having n possible positions, a point can be shifted n times without repeating its position only if the shifting step

and the number of available positions are coprime. This property can be generalized to any number of points by introducing the concepts of necklaces and symmetry of a necklace.

Let $\mathcal{G}$ be a necklace. The symmetry of a necklace [14], $Sym(\mathcal{G})$, is defined as the minimum value of r such that:

$$Sym(\mathcal{G}) = \min\{1 \leq r \leq n : \mathcal{G} + r \equiv \mathcal{G} \bmod(n)\}. \quad (9)$$

As an example, an image of six stars is taken ($n = 6$) and we define the necklace $\mathcal{G} = \{1, 3, 5\}$ (see Fig. 3). For this particular case, $Sym(\mathcal{G}) = 2$ because $\{1, 3, 5\} \equiv \{3, 5, 7\} \bmod 6$. This, in fact, leads to the same combination of observed stars, so the symmetry gives the idea of how many steps are required to complete a cycle in the modular arithmetic during the shifting.

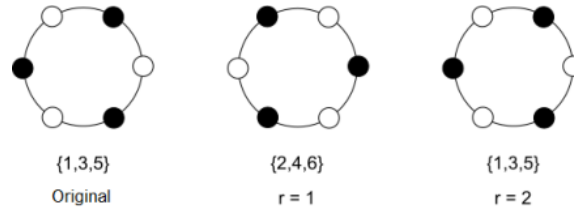


Figure 3: Symmetry of a necklace.

Therefore, the number of steps that a necklace is able to shift with no repetition can be obtained by the following relation:

$$n_{step} = \frac{Sym(\mathcal{G})}{\gcd(d_{step}, Sym(\mathcal{G}))}, \quad (10)$$

where $\gcd(d_{step}, Sym(\mathcal{G}))$ is the greatest common divisor between d_{step} and $Sym(\mathcal{G})$. In the algorithm of SN, this condition is checked by the analysis of the values of $\delta\mathcal{G}(i)$, which is a faster process.

4 Enhanced Shifting Necklaces

Enhanced Shifting Necklaces (ESN) is a modification on the original Shifting Necklaces algorithm where instead of fixing a pattern that is shifted during the sequence, the pattern is changed in each step of the sequence. This provokes that in each step two different movements are required, one inside the available positions (as the SN algorithm), and the other over the possible necklaces. As in the case of Shifting Necklace algorithm, the pattern used is a necklace that is generated using the methodology shown before.

The process of generating the sequence is similar as in the SN algorithm, but with three important differences. First, a separation between the necklaces that present symmetries and the ones that do not is performed. This

separation is done in order to simplify the generation, as necklaces with symmetries cannot be shifted the same number of times that the other cases (see Eq. (10)). That way, for the case of triads, there is one case of symmetry $Sym(\mathcal{G}) = n/3$ when $\delta\mathcal{G}(1) = \delta\mathcal{G}(2) = \delta\mathcal{G}(3)$. On the other hand and for the case of quads, there are two: $Sym(\mathcal{G}) = n/2$ when $\delta\mathcal{G}(1) = \delta\mathcal{G}(3)$ and $\delta\mathcal{G}(2) = \delta\mathcal{G}(4)$, and $Sym(\mathcal{G}) = n/4$ when $\delta\mathcal{G}(1) = \delta\mathcal{G}(2) = \delta\mathcal{G}(3) = \delta\mathcal{G}(4)$.

Second, the shifting step is left free as a parameter of the algorithm and constant for all the necklaces. This has two purposes, to make the parameter independent of the necklace (and thus improving the performance speed of the algorithm) and to have a parameter that will be used later to optimize the algorithm under the *expected time to discovery* metric [7] for a given number of observed stars. That way, the shifting step has been optimized for the most common values of observed stars. In other cases, we take $d_{step} = 1$ as a default value for any number of stars. For completeness, Table 1 includes the optimal shifting step for triads and quads up to $n = 20$ stars.

Table 1: Optimal shifting step for triads and quads up to $n = 20$ stars.

n	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
triad	1	4	5	6	5	2	3	7	5	4	9	11	5	7	11	16	17
quad	1	1	5	6	7	7	3	10	7	12	13	11	5	8	5	17	13

Third, the computation of the next kernel in the sequence has to be done in two steps as the necklace in this case is not fixed. In order to do that, we create a lattice with the necklaces that present no symmetries and, in each step of the sequence, a movement in this lattice is performed, obtaining a different necklace. Moreover, as resonances can appear between the movement in the available positions and the movement in the lattice, an additional movement in the lattice of necklaces is required when this condition is fulfilled:

$$s \bmod (\text{lcm}(n, n_l)) = 0, \quad (11)$$

where s is the position of the kernel in the sequence, and $\text{lcm}(n, n_l)$ is the least common multiplier between the number of observed stars n and the number of necklaces in the lattice defined n_l . As it can be observed, this requires to compute first the new necklace in the lattice and then applied it with the shifting step to the former configuration, in particular, the last index of the previous kernel is taken as a reference for the application of $\delta\mathcal{G}(i)$, that is:

$$\begin{aligned} \mathcal{G}_{s+1}(\kappa) &= \mathcal{G}_s(\kappa) + d_{step} \bmod (n), \\ \mathcal{G}_{s+1}(i) &= \mathcal{G}_{s+1}(i+1) - \delta\mathcal{G}_{s+1}(i) \bmod (n) \quad \forall i < \kappa, \end{aligned} \quad (12)$$

where $\mathcal{G}_s(i)$ represents the position i inside the necklace s of the sequence.

This process is repeated until all necklaces have appeared a number of times equal to n_{step} , obtained from Eq. (10).

5 Robustness of kernel generators

In this section, an assessment of Lexicographic ordering (Lex), Pattern Shifting (PS), Enhanced Pattern Shifting (EPS), Shifting Necklaces (SN), Enhanced Shifting Necklaces (ESN), and Base Unrank (BU) kernel generators is presented, according to four robustness metrics. The EPS, SN, and ESN are the new algorithms introduced with this paper. The BU algorithm was introduced in [7] and was chosen among all the algorithms presented in [7] due to its performance in terms of speed and in terms of its *expected time to discovery* score.

The metrics used to evaluate the algorithms are the following: 1) the *worst-case failure count*; 2) the *worst-case failure count sum*; 3) the *maximum difference of index distribution*; and 4) the *expected time to discovery*. The first three metrics are introduced here whereas the *expected time to discovery* was introduced in [7].

5.1 Worst-case failure count

Given the number of observed stars n and false stars f of an image, the *worst-case failure count* of a sequence of kernels is computed by searching for the worst combination of f false stars (worst scene) that will maximize the number of kernels used before a kernel composed only of actual stars is found. In other words, this metric counts the maximum number of bad kernels that can be returned using that sequence before the first good kernel is found.

This metric provides an upper bound on the number of iterations required to discover a good kernel. From this number, the time allocated to the Star-ID process is derived. The value of this time is particularly important for hard real time systems.

The number of distinct kernels of size κ ($\kappa = 3$ for triads and 4 for quads) that can be produced from n observations is:

$$N = \binom{n}{\kappa} = \frac{n!}{\kappa!(n-\kappa)!}. \quad (13)$$

which is also the size of any complete sequence without repetition, as the ones generated by the algorithms studied here. When $f = 0$, the *worst-case failure count* is zero for any sequence, since, in that case, the first kernel selected will always be a good one. Conversely, when $f = n - \kappa$ there will exist just one good kernel, in which case the value of this metric will be $N - 1$ for all complete sequences without repetitions. For $0 < f < n - \kappa$

the value of this metric will depend on the quality of the sequence. Fig. 4 shows the results of this metric for triads when $n = 14$ (left) and for quads when $n = 9$ (right).

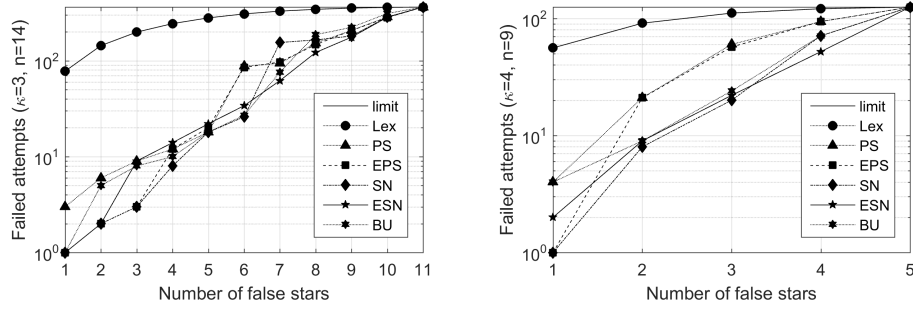


Figure 4: Worst-case failure count for triads with $n = 14$ (left) and for quads with $n = 9$ (right).

Among all possible complete sequences without repetition, the worst performing sequences are those that for a particular combination of false stars have all bad kernels at the beginning and all good kernels at the end. The *worst-case failure count* for such sequences will be:

$$N_{wcs} = N - N_g \quad (14)$$

where:

$$N_g = \binom{g}{\kappa} = \frac{g!}{\kappa! (g - \kappa)!} \quad (15)$$

with $g = f - \kappa$ being the number of actual stars. This upper limit on bad performance is indicated on Fig. 4. Lex algorithm always attains this limit, making it a very bad choice.

5.2 Worst-case failure count sum

The *worst-case failure count* metric can be cumbersome for comparing different algorithms due to its dependence on the number of false stars f , which leads to a very large number of combinations on the number of false stars and number of observed stars that have to be studied to get a complete picture. A more compact metric can be obtained by summing the results of all possible combinations of false stars for each particular number of observed stars n , as follows:

$$N_{wcfcs}(Q, n) = \sum_{f=0}^{n-\kappa} N_{wfc}(Q, n, f) \quad (16)$$

where $N_{wfc}(Q, n, f)$ is the *worst-case failure count* for n observed stars and f false stars, introduced in the previous section, and Q is the complete

sequence of kernels generated by the algorithm being evaluated. The results of this metric for triads are shown in Table 2 and for quads in Table 3.

Table 2: Results for the *worst-case failure count sum* metric ($\kappa = 3$).

n	Lex	PS	EPS	SN	ESN	BU
4	3	3	3	3	3	3
5	15	12	12	11	11	11
6	45	34	32	30	30	35
7	105	66	65	54	55	67
8	210	128	123	114	101	120
9	378	198	192	178	168	184
10	630	321	311	300	298	346
11	990	464	454	414	444	483
12	1485	670	660	676	580	655
13	2145	878	867	865	848	865
14	3003	1239	1219	1208	1089	1229
15	4095	1564	1546	1588	1375	1558
16	5460	1999	1975	2158	1688	1958
17	7140	2520	2494	2564	2258	2422
18	9180	3192	3156	3448	2795	3533
19	11628	3781	3755	3990	3276	4229
20	14535	4695	4642	5178	3825	4997

Table 3: Results for the *worst-case failure count sum* metric ($\kappa = 4$).

n	Lex	PS	EPS	SN	ESN	BU
5	4	4	4	4	4	4
6	24	20	20	16	18	19
7	84	60	60	49	52	59
8	224	143	140	107	115	123
9	504	304	298	225	210	233
10	1008	544	544	503	394	414
11	1848	950	941	782	709	740
12	3168	1528	1518	1451	1143	1154
13	5148	2398	2380	2085	1814	1830
14	8008	3434	3429	3548	2614	2647
15	12012	5083	5063	4968	3510	4167
16	17472	7056	7035	7534	4993	5585
17	24752	9538	9516	9745	6492	7483
18	34272	12736	12709	14370	9028	10161
19	46512	17056	17024	18142	11481	13921
20	62016	21503	21473	24994	15219	17600

As it can be seen, ESN is the algorithm that performs, in general, better under this metric. This is more relevant as the number of observed stars increases, while for low number of observed stars, SN performs slightly better in some particular cases. For $n \geq 2\kappa$, EPS always presents better results than PS.

5.3 Maximum difference of index distribution

The *maximum difference of index distribution* provides a measurement of how well the usage of observed stars is kept balanced during the sequence

generation. Algorithms that perform a more uniform distribution of indexes in their sequences over the observed stars present a good score under this metric, whereas those that continuously select the same stars along the sequence have a bad score (larger values).

This metric is computed as follows: a n -long vector of counters is initialized with zeros, where each counter i will count the number of times that each observed star i has been used. Then, at each step, one kernel is read from the sequence and the counters associated with the observed stars in that kernel are updated. At the end of each step, the difference (dif) between the maximum and minimum counts is computed. The maximum value of (dif) ever found after the whole sequence has been processed is the value of this metric. For example, for EPS with $n = 7$ and $\kappa = 3$, the value of this metric is 6, since at the 23rd kernel processed, the third observed star has been used 13 times and the seventh observed star has been used only 7 times, and the difference $6 = 13 - 7$ is the largest difference ever found between all counters while processing the sequence.

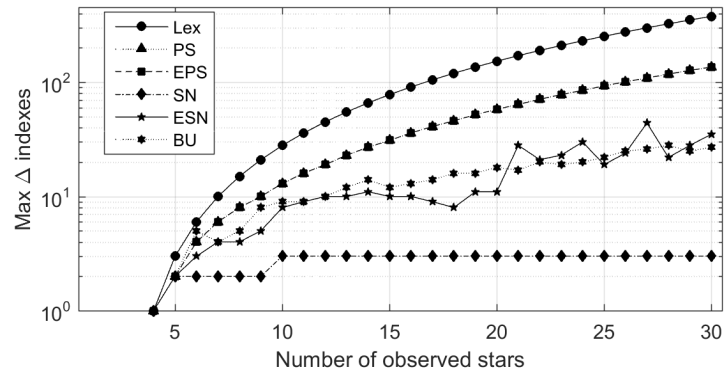


Figure 5: *Maximum difference of index distribution for triads*

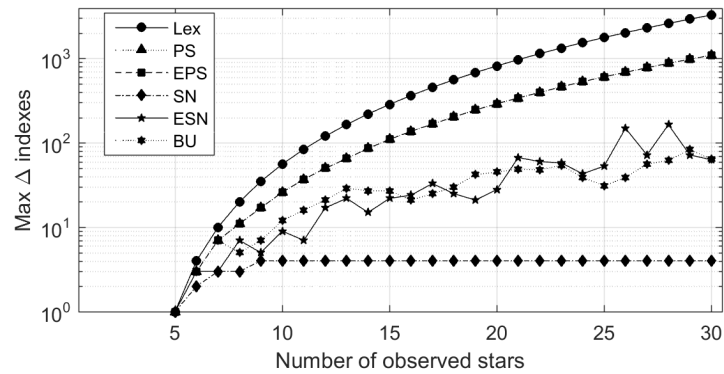


Figure 6: *Maximum difference of index distribution for quads*

Figure 5 shows the results of the *maximum difference of index distribution* (Max Δ indexes) for triads and Fig. 6 for quads. As it can be seen, SN presents the highest uniformity in all the sequences generated, followed by ESN and BU. EPS and PS follows next, with the same scores under this metric. Finally, Lex has the worst scores, as expected.

This metric is fast to compute. In fact, the complete sequence of kernels does not even need to be stored, as this metric can be computed using kernels generated on the fly. On the other hand, the usefulness of this metric is that it allows to quickly select promising candidates to be tested with more costly metrics, like the *worst-case failure count sum* and *expected time to discovery*, since algorithms that have a good score with it also tend to have good scores with the other two metrics (ESN and BU being two examples). It should be stressed, however, that this metric should not be used as a replacement to the others. For example, SN has the best score in this metric, but performs not so well regarding the other two.

5.4 Expected time to discovery

The *expected time to discovery* is a metric introduced in [7]. This metric computes the average number of kernels that have to be generated in order to get the first good kernel (a kernel made of actual stars, only), under the assumption that all scenes (all possible combinations of real and false stars) are equally probable, whenever a solution is possible, that is, having at least κ real stars. It is important to notice that the lower the score of this metric, the better, as the algorithm requires, on average, less attempts to identify a scene.

Let Q be the complete sequence of kernels provided by an algorithm and let s be a possible scene where $s \in S$ is the set of possible scenes. The *expected time to discovery*, $T(Q)$, according to [7], is defined as:

$$T(Q) = \frac{1}{|S|} \sum_{s \in S} \tau(Q, s); \quad (17)$$

where $\tau(Q, s)$ represents the number of kernels that the algorithm must generate to discover the scene s , and $|S| = \sum_{t=\kappa}^n \binom{n}{t}$ is the number of possible scenes where a solution is possible.

Tables 4 and 5 show the values of $T(Q)$ for Lex, PS, EPS, SN, ESN, and BU algorithms. As can be seen, both for triads and quads, ESN and BU have the best performance, followed by SN, EPS, and then by PS. As expected, Lex performs the worst.

Table 4: Results for the *expected time to discovery* metric ($\kappa = 3$).

n	Lex	PS	EPS	SN	ESN	BU
4	2.20	2.20	2.20	2.20	2.20	2.20
5	4.44	4.19	4.19	4.06	4.06	4.13
6	7.88	6.86	6.74	6.69	6.64	6.81
7	12.61	9.89	9.74	9.36	9.15	9.55
8	18.61	13.01	12.85	12.36	11.81	12.44
9	25.85	15.91	15.62	15.16	14.47	14.87
10	34.23	18.37	18.03	17.41	16.77	17.21
11	43.68	20.26	19.91	19.20	18.20	18.03
12	54.15	21.55	21.04	20.52	18.76	18.72
13	65.57	22.28	21.72	21.28	18.80	18.72
14	77.93	22.54	21.98	21.53	18.81	18.81
15	91.23	22.42	21.67	21.61	17.93	17.97
16	105.45	22.04	21.24	21.14	17.25	17.27
17	120.62	21.48	20.68	20.55	16.96	16.66
18	136.74	20.81	19.80	19.93	16.17	16.42
19	153.83	20.10	19.05	19.19	15.39	15.51
20	171.89	19.38	18.33	18.36	14.52	14.74

Table 5: Results for the *expected time to discovery* metric ($\kappa = 4$).

n	Lex	PS	EPS	SN	ESN	BU
5	2.67	2.67	2.67	2.67	2.67	2.67
6	6.68	6.32	6.32	6.05	6.14	6.18
7	14.34	12.52	12.52	11.97	11.69	12.13
8	27.03	21.56	21.45	19.91	19.44	19.69
9	46.10	33.25	33.11	30.54	28.41	29.12
10	72.81	47.04	46.86	42.75	39.54	40.07
11	108.28	62.04	61.86	55.80	50.38	51.61
12	153.49	77.25	76.94	68.88	60.31	60.22
13	209.32	91.69	91.35	81.28	66.85	67.56
14	276.59	104.58	104.21	92.40	73.06	71.67
15	356.08	115.40	115.02	101.37	75.14	74.98
16	448.63	123.89	123.35	108.54	77.43	74.61
17	555.10	130.05	129.47	113.16	75.45	74.06
18	676.41	134.04	133.42	115.83	73.73	72.26
19	813.52	136.13	135.52	116.76	69.15	70.62
20	967.42	136.64	135.82	116.54	66.92	66.52

6 Speed

6.1 Method

To take speed measurements, a single threaded program was written in C language². This program was compiled with gcc 4.9.3 for mingw32 target, with optimization switch -O2. All tests were performed on an Intel i5-2467M processor running at 1.6 GHz, with 8 GB of RAM and Windows 7. As the program was single threaded, it made use of only one CPU core out of the many present in the i5 processor. To measure loop and control structures overhead, an empty *dummy* code (executing no statement)

²Full source code of this program is available at:
<http://mortari.tamu.edu/Attitude-Estimation/TriadQuad.zip>

was also tested and the run time measurements obtained with it were subtracted from raw measurements of other algorithms in order to get a value closer to their real execution time. Time measurements were taken using the *QueryPerformanceCounter* function from the Windows API, which in the test machine had a resolution of about 0.6 microseconds. To smooth out short term fluctuations, the algorithms were executed thousands of times and their measured execution time averaged. To monitor possible longer term fluctuations the whole test was repeated three times. In order to reduce interference from other processes, most processes and applications in the machine were shut down before taking the measurements. The following algorithms were tested: Lex, PS, EPS, SN, ESN, and BU (Base Unrank over revolving door). Results are shown in Fig. 7. Each point plotted in this figure was obtained by running the sequence generating algorithms 20,000 times for triads and 3,000 times for quads and averaging the results. For each algorithm three curves are plotted, each corresponding to each run.

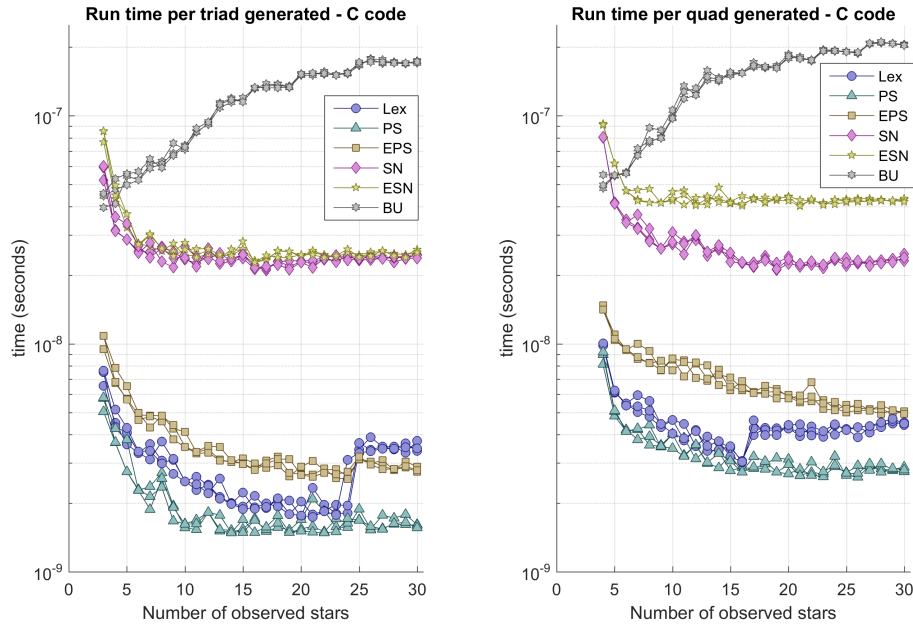


Figure 7: Average time spent per kernel generated, for $n \in [3, 30]$

6.2 Discussion

Short term time fluctuations (in the order of nanoseconds and microseconds) can be explained by the resolution of the timer used (about $0.6 \mu\text{s}$) and the practically non deterministic run time behavior in modern processors (this has many causes, one of them being associated with the occurrence of cache memory misses, some of which may be induced by other processes running

in parallel in other CPU cores). Long term time fluctuations (in the order of some milliseconds) arise mainly from long interruptions that may be caused by other higher priority background processes, operating system drivers, and hard disk operations.

As it can be seen in Fig. 7, all kernel generators spent no more than 178 nanoseconds per triad generated. Considering that the clock frequency of the processor is 1.6 GHz, this means less than 285 clock cycles per triad generated, even for the slowest case of $n = 26$ and the slowest algorithm (BU). The fastest algorithm in this test was PS. For $n \geq 15$, PS spent on average 1.6 nanoseconds to generate each triad. This equates to about 2.6 clock cycles per triad generated. For quads, the results were similar, with 211.4 ns (338 clock cycles) taken by BU for $n = 28$ in the slowest of the three tests, and less than 3 ns (4.8 clock cycles) taken by PS on average for $n > 15$. Overall, as the number of observed stars n increases, the algorithms become faster, with the exception of BU. This happens because as n increase, the effect of loop overhead inside the algorithms become less important.

Base-unrank performed the worst, even with the use of a 820 byte look-up table of binomial coefficients to speed up the algorithm. A possible explanation is that computing the unranking function for the revolving door sequence (the sequence on which our and Mueller et al. [7] implementation of BU was based) is not trivial. It should be noted that our implementation of BU was not fully optimized. We have opted to keep the general form of the algorithm for any kernel size, basing our code in the code provided by authors of [7], instead of spending time and effort in coding an specialized version of BU for triads and quads. The version of BU shown in Fig. 7 makes use of this 820 byte look-up table. Without it, BU runs about three times slower.

Overall, for $n > 5$, the fastest algorithm in our tests was PS, followed by Lex, EPS, SN, ESN, and BU, in that order. For $n > 5$, Lex, PS, and EPS were able to generate more than 200 million triads per second and more than 100 million quads per second. SN and ESN were slower by about one order of magnitude, generating about 40 million triads per second for $n > 6$ and more than 20 million quads per second for $n > 5$. However, the SN and ESN algorithms have not been devised for speed but for robustness and our implementation was not fully optimized. With some effort it might be possible to find faster implementations for SN and ESN.

Based on algorithm complexity, Lex was expected to be the fastest algorithm. However, for some unknown reason, PS performed faster than Lex in all cases, albeit being slightly more complex. We also observed a significant increase in the time spent per triad in Lex for $n \geq 25$. The causes for these unexpected results are still unknown, but we suspect that they might be related to the occurrence of cache misses. Only a detailed analysis involving the object code generated and microarchitecture details of the CPU used in the tests could offer a definitive explanation. It was noted, however, that

if the program is compiled with no optimization, then Lex runs faster than PS, as it would be expected.

The values of run time for all kernel generator algorithms tested is much shorter than the typical run time of the fastest Star-ID algorithms (e.g., Pyramid [6]). In a test run performed with Pyramid on the same machine, using typical parameters of a wide field of view star tracker, Pyramid took on average $43\ \mu\text{s}$ to identify a scene [15]. This means that speed should not be the primary criterion of selection among PS, EPS, SN, ESN, and BU, as their run time probably has little effect on the total time spent in the Star-ID process. It should be noted, however, that to be considered for implementation in a star tracker, any kernel generator has to be able to generate a new kernel in a much shorter time than the time spent by the Star-ID algorithm. This precludes the on-board implementation of most other algorithms proposed in [7].

A hybrid implementation, that reads the sequences of kernels from a small look-up table for small n and uses one of the algorithms proposed here for larger n seems to be a very promising approach for a star tracker. Such implementation would benefit from optimally generated sequences of kernels for small n – these sequences could be generated by the best algorithms proposed by [7] – while still keeping most of the speed advantages of the algorithms proposed in this work. The memory requirements for this look-up table are very modest. For example, to store all sequences of triads with $n \leq 12$ or all sequences of quads with $n \leq 11$, less than 1.6 kilobyte is required.

7 Conclusions

This work introduces three new kernel generator algorithms, Enhanced Pattern Shifting (EPS), Shifting Necklaces (SN) and Enhanced Shifting Necklaces (ESN), and compares them with the original Pattern Shifting (PS), Base Unrank (BU) and the Lexicographic (Lex) algorithms. In addition, new robustness metrics are introduced, complementing the *expected time to discovery* metric introduced in [7].

Regarding speed, PS, Lex and EPS are the fastest algorithms, with the time spent per kernel generated comparable to the time spent by branching/jumping instructions in modern processors. SN and ESN are also very fast, although about an order of magnitude slower. BU was the slowest algorithm, but this might not make justice to it, since its source code was not fully optimized for speed. Anyway, it is expected that the run time of algorithms evaluated in this work will have little importance when selecting an algorithm for implementation in a star tracker, being their robustness and code complexity much more important.

Robustness metrics were tested up to $n \leq 20$. ESN is the best algorithm

according to the *worst-case failure count sum*, being outperformed by SN in rare cases. According to the *expected time to discovery* metrics ESN and BU are the most robust algorithms, having similar performances.

In terms of code complexity, PS and EPS are the simplest algorithms, each taking about 15 to 20 lines of code when written in C. This compares to about 100 lines of code for each of SN and ESN and about 80 lines of code for BU implementations for triads, which it is still reasonable. However, in critical mission designs, where the code complexity of SN, ESN and BU might be considered a factor against them, EPS provides a good alternative. Moreover, EPS provides more robust kernel sequences than PS, with very little complexity added and a negligible run time overhead.

Lex, PS and SN are not recommended for new designs, since there are much better algorithms (EPS, ESN and BU) with similar speed, performance and code complexity, with much better robustness. BU has about the same robustness of ESN, but is significantly slower than ESN. Unless a faster implementation of BU is found, or BU is proved to be better than ESN for $n > 20$, ESN would be preferred.

Unfortunately, due to the high computational cost of the *worst-case failure count sum* and *expected time to discovery* metrics, we were unable to evaluate the algorithms for $n > 20$ using these metrics. It would be interesting in the future to expand the analysis for $n > 20$ and also to evaluate new algorithms.

A very promising idea is to combine the power of optimal or near optimal solutions generated on the ground for small n , with one of the best evaluated algorithms in this paper (EPS and ESN) for larger n . This idea should be further investigated in the future.

8 Acknowledgments

The work of the first author was supported by the Spanish Ministry of Economy and Competitiveness (Project no. ESP2013-44217-R) and by the research project CUD2015-02 at Centro Universitario de la Defensa, Zaragoza.

The second author would like to thank CNPq (Brazilian National Council for Scientific Research) and AEB (Brazilian Space Agency) for the PhD interchange scholarship grant, and also for the many individuals and organizations who made this research possible.

We are grateful for all the reviewers of this paper for their time spent helping us improve this work. We wish also to say thanks to the authors of [7] who kindly provided us their code of Base-unrank algorithm on which we based our implementation.

References

- [1] LIEBE, C.C., *Accuracy Performance of Star Trackers: a Tutorial.*, IEEE Transactions on Aerospace and Electronic Systems 38 (2) (2000) 587-599. doi: 10.1109/TAES.2002.1008988.
- [2] ENRIGHT, J., SINCLAIR, D., CORDELL GRANT, C., MCVITTIE G., DZAMBA, T., *Towards Star Tracker Only Attitude Estimation*, 24th Annual AIAA/USU Conference on Small Satellites (2010) Paper SSC10-X-3.
- [3] GOTTLIEB, D.M. *Angular Separation Match Technique*, Spacecraft Attitude Determination and Control, edited by J. R. Wertz, Reidel, Dordrecht, The Netherlands, (1978) 262-263.
- [4] KOSIC, J.C. *Star Pattern Identification Aboard an Inertially Stabilized Spacecraft*. Journal of Guidance, Control, and Dynamics 14 (2) (1991) 230-235. doi: 10.2514/3.20632.
- [5] HO, K. *A survey of algorithms for star identification with low-cost star trackers*. ACTA Astronautica 73 (April-May) (2012) 156-163. doi: 10.1016/j.actaastro.2011.10.017.
- [6] MORTARI, D., SAMAAN, M.A., BRUCCOLERI, C., JUNKINS, J.L., *The Pyramid Star Identification Technique*, Navigation 51 (3) (2004) 171-183. doi: 10.1002/j.2161-4296.2004.tb00349.x.
- [7] MUELLER, J.H., SÁNCHEZ-SÁNCHEZ, C., SIMÕES, L.F., IZZO, D., *Optimal Orderings of k-subsets for Star Identification*, arXiv preprint (2016) arXiv:1607.04552.
- [8] SAMAAN, M., THEIL, S. *Development of a Low Cost Star Tracker for the SHEFEX Mission*, Aerospace Science and Technology 23 (1) (2012) 469-478. doi: 10.1016/j.ast.2011.09.013.
- [9] SEGERT, T. *Development of the Pico Star Tracker ST-200 – Design Challenges and Road Ahead*, 25th Annual AIAA/USU Conference on Small Satellites (2011) Paper SSC11-IX-4.
- [10] KEYS, A.S., ADAMS, J.H., FRAZIER, D.O, PATRICK, M.C., WATSON, M.D. *Developments in Radiation-Hardened Electronics Applicable to the Vision for Space Exploration* AIAA SPACE 2007 Conference & Exposition (2007) Paper AIAA 2007-6269. doi: 10.2514/6.2007-6269.
- [11] NASA - NATIONAL AERONAUTICS AND SPACE ADMINISTRATION *Radiation Resistant Computers*, webpage (2005). http://science.nasa.gov/science-news/science-at-nasa/2005/18nov_eaftc/, last accessed on August 8, 2016.

-
- [12] KUMAR, M., MORTARI, D., JUNKINS, J.L. *Analytical Approach to Star Identification Reliability*, ACTA Astronautica 66 (3-4) (2010) 508-515. doi: 10.1016/j.actaastro.2009.07.005.
- [13] CATTELL, K., RUSKEY, F., SAWADA, J., SERRA, M., MIERS, R., *Fast Algorithms to Generate Necklaces, Unlabeled Necklaces, and Irreducible Polynomials over $GF(2)$* , Journal of Algorithms 37 (2) (2000) 267-282. doi: 10.1006/jagm.2000.1108.
- [14] CASANOVA, D., AVENDAÑO, M.E., MORTARI, D., *Design of Flower Constellations using Necklaces*, IEEE Transactions on Aerospace and Electronic Systems 50 (2) (2014) 1347-1358. doi: 10.1109/TAES.2014.120269.
- [15] FIALHO, M.A.A., MORTARI, D., PERONDI, L.F. *Performance Tests of the Pyramid Star-ID Algorithm with Memory Adaptive K-Vector*. To appear on 27th AAS/AIAA Space Flight Mechanics Meeting, San Antonio, Texas, 2017.

A Pseudo-code listing for quad version of EPS

Algorithm 4: Quad generator using Enhanced Pattern Shifting

```

for  $dj$  from 1 to  $(n-3)$  do
  for  $dk$  from 1 to  $(n-dj-2)$  do
    for  $dr$  from 1 to  $(n-dj-dk-1)$  do
      for  $ii$  from 1 to 4 do
        for  $i$  from  $ii$  to  $(n-dj-dk)$  in steps of 4 do
           $j = i + dj;$ 
           $k = j + dk;$ 
           $r = k + dr;$ 
          next kernel is “[ $i, j, k, r$ ]”;
        end
      end
    end
  end
end

```

B Pseudo-code listings for Shifting Necklaces

Algorithm 5: Triad generator using Shifting Necklaces

```

 $n_{max} = \text{floor}(n/3);$ 
 $q = 3;$ 
for  $\delta\mathcal{G}(1)$  from 1 to  $n_{max}$  do
    while  $(2\delta\mathcal{G}(2) \leq n - \delta\mathcal{G}(1))$  do
        for  $p$  from 1 to  $(n - 1)$  do
            if  $(p \neq \delta\mathcal{G}(1)).\text{and.}(p \neq \delta\mathcal{G}(2)).\text{and.}(p \neq n - \delta\mathcal{G}(1) - \delta\mathcal{G}(2)).\text{and.}(p \neq$ 
 $n - \delta\mathcal{G}(1)).\text{and.}(p \neq n - \delta\mathcal{G}(2)).\text{and.}(p \neq \delta\mathcal{G}(1) + \delta\mathcal{G}(2))$  then
                 $q = p;$ 
                if  $(\text{gcd}(p, n) == 1)$  then
                     $d_{step} = p - 1;$  break;
                end
            end
        end
         $flag = false;$ 
         $n_3 = n;$ 
        if  $(p == n - 1)$  then
             $dstep = q - 1;$ 
             $n_3 = \text{gcd}(q, n);$ 
             $flag = true;$ 
        end
         $[i, j, k] = [1, 1 + \delta\mathcal{G}(1), 1 + \delta\mathcal{G}(1) + \delta\mathcal{G}(2)];$ 
        next kernel is “[i, j, k]”;
        if  $(\delta\mathcal{G}(1) == \delta\mathcal{G}(2)).\text{and.}(n/3 == \delta\mathcal{G}(2))$  then
             $n_2 = \delta\mathcal{G}(2);$ 
        else
             $n_2 = n;$ 
        end
        for  $m$  from 2 to  $n_2$  do
            if  $(\text{mod}(m - 1, n_2/n_3) == 0).\text{and.}(flag == true)$  then
                 $[i, j, k] = [i, j, k] + 1;$ 
            end
             $[i, j, k] = \text{mod}([i, j, k] + dstep, n) + 1;$ 
            next kernel is “[i, j, k]”;
        end
        if  $(\delta\mathcal{G}(1) == \delta\mathcal{G}(2)).\text{or.}(2\delta\mathcal{G}(2) == n - \delta\mathcal{G}(1))$  then
            else
                 $[i, j, k] = [1, 1 + \delta\mathcal{G}(2), 1 + \delta\mathcal{G}(2) + \delta\mathcal{G}(1)];$ 
                next kernel is “[i, j, k]”;
                for  $m$  from 2 to  $n$  do
                    if  $(\text{mod}(m - 1, n/n_3) == 0).\text{and.}(flag == true)$  then
                         $[i, j, k] = [i, j, k] + 1;$ 
                    end
                     $[i, j, k] = \text{mod}([i, j, k] + dstep, n) + 1;$ 
                    next kernel is “[i, j, k]”;
                end
            end
        end
         $\delta\mathcal{G}(2) = \delta\mathcal{G}(2) + 1;$ 
    end
end

```

Algorithm 6: Quad generator using Shifting Necklaces

```

 $n_{max} = \text{floor}(n/4)$ 
 $v = 4;$ 
for  $\delta\mathcal{G}(1)$  from 1 to  $n_{max}$  do
  for  $\delta\mathcal{G}(2)$  from  $\delta\mathcal{G}(1)$  to  $\text{floor}((n - \delta\mathcal{G}(1))/3)$  do
     $\delta\mathcal{G}(3) = \delta\mathcal{G}(2);$ 
    while  $(2\delta\mathcal{G}(3) \leq n - \delta\mathcal{G}(1) - \delta\mathcal{G}(2))$  do
      for  $u$  from 1 to  $(n - 1)$  do
        if  $(u \neq \delta\mathcal{G}(1)).\text{and.}(u \neq \delta\mathcal{G}(2)).\text{and.}(u \neq \delta\mathcal{G}(3)).\text{and.}(u \neq$ 
           $n - \delta\mathcal{G}(1) - \delta\mathcal{G}(2) - \delta\mathcal{G}(3)).\text{and.}(u \neq \delta\mathcal{G}(1) + \delta\mathcal{G}(2)).\text{and.}(u \neq$ 
           $\delta\mathcal{G}(1) + \delta\mathcal{G}(3)).\text{and.}(u \neq n - \delta\mathcal{G}(2) - \delta\mathcal{G}(3)).\text{and.}(u \neq$ 
           $\delta\mathcal{G}(2) + \delta\mathcal{G}(3)).\text{and.}(u \neq n - \delta\mathcal{G}(1) - \delta\mathcal{G}(3)).\text{and.}(u \neq$ 
           $n - \delta\mathcal{G}(1) - \delta\mathcal{G}(2)).\text{and.}(u \neq \delta\mathcal{G}(1) + \delta\mathcal{G}(2) + \delta\mathcal{G}(3)).\text{and.}(u \neq$ 
           $n - \delta\mathcal{G}(2)).\text{and.}(u \neq n - \delta\mathcal{G}(3)).\text{and.}(u \neq n - \delta\mathcal{G}(1))$  then
           $v = u;$ 
          if  $(\text{gcd}(u, n) = 1)$  then
             $d_{step} = u - 1;$  break
          end
        end
      end
      end
       $flag = false;$ 
       $n_3 = n;$ 
      if  $(u = n - 1)$  then
         $d_{step} = v - 1;$ 
         $n_3 = \text{gcd}(v, n);$ 
         $flag = true;$ 
      end
       $[i, j, k, r] = [1, i + \delta\mathcal{G}(1), j + \delta\mathcal{G}(2), k + \delta\mathcal{G}(3)];$ 
      next kernel is “[i, j, k, r]”;
      if  $(\delta\mathcal{G}(1) = \delta\mathcal{G}(2)).\text{and.}(\delta\mathcal{G}(2) = \delta\mathcal{G}(3)).\text{and.}(\delta\mathcal{G}(3) = n/4)$ 
        then
           $n_2 = \delta\mathcal{G}(3);$ 
        else if  $(\delta\mathcal{G}(1) = \delta\mathcal{G}(3)).\text{and.}(2\delta\mathcal{G}(2) = n - \delta\mathcal{G}(1) - \delta\mathcal{G}(3))$  then
           $n_2 = n/2;$ 
        else
           $n_2 = n;$ 
        end
        for  $s$  from 2 to  $n_2$  do
          if  $(\text{mod}(s - 1, n_2/n_3) == 0).\text{and.}(flag == true)$  then
             $[i, j, k, r] = [i, j, k, r] + 1;$ 
          end
           $[i, j, k, r] = \text{mod}([i, j, k, r] + d_{step}, n) + 1;$ 
          next kernel is “[i, j, k, r]”;
        end
        ... It continues with all permutations of  $\delta\mathcal{G}(1)$ ,  $\delta\mathcal{G}(2)$  and  $\delta\mathcal{G}(3)$ .
      end
    end
  end
end

```

C Pseudo-code listings for Enhanced Shifting Necklaces (triads)

Algorithm 7: Triad generator using Enhanced Shifting Necklaces

```

if  $\text{mod}(n, 3) == 0$  then
  |  $nmax = n/3 - 1$ ;  $des = n/3$ ;
else
  |  $nmax = \text{floor}(n/3)$ ;  $des = 0$ ;
end
 $cq = [0, 0, 0, 0, 3, 4, 5, 4, 1, 2, 6, 4, 3, 8, 10, 4, 6, 10, 15, 16]$ ;
if  $n \leq 20$  then
  |  $q = cq(n)$ ;
else
  |  $q = 0$ ;
end
 $di = 1$ ;  $dj = 1$ ;  $k = 2 - q$ ;  $c = 0$ ;  $nmax3 = \text{floor}((n - di)/2)$ ;
for  $k1$  from 1 to  $nmax$  do
  |  $nmax2 = \text{floor}((n - k1)/2)$ ;
  | for  $k2$  from  $k1$  to  $nmax2$  do
    |  $c = c + 1$ ;  $k = \text{mod}(k + q, n) + 1$ ;  $j = \text{mod}(k - dj - 1, n) + 1$ ;
    |  $i = \text{mod}(j - di - 1, n) + 1$ ;
    | next kernel is “[i, j, k]”;
    |  $[di, dj, nmax3] = \text{nextnecklace}(di, dj, nmax3, nmax, n)$ ;
    | if  $di \neq dj$  and  $2dj \neq n - di$  then
      |  $c = c + 1$ ;  $k = \text{mod}(k + q, n) + 1$ ;  $j = \text{mod}(k - dj - 1, n) + 1$ ;
      |  $i = \text{mod}(j - di - 1, n) + 1$ ;
      | next kernel is “[i, j, k]”;
      |  $[di, dj, nmax3] = \text{nextnecklace}(di, dj, nmax3, nmax, n)$ ;
    | end
  | end
end
if  $n \neq 3$  then
  |  $cmax = c$ ;  $par = \text{lcm}(n, cmax)$ ;
  | if  $par == cmax$  then
    |  $[di, dj, nmax3] = \text{nextnecklace}(di, dj, nmax3, nmax, n)$ ;
  | end
end
while  $c \neq nc-des$  do
  |  $c = c + 1$ ;  $k = \text{mod}(k + q, n) + 1$ ;  $j = \text{mod}(k - dj - 1, n) + 1$ ;
  |  $i = \text{mod}(j - di - 1, n) + 1$ ;
  | next kernel is “[i, j, k]”;
  | if  $\text{mod}(c, par) == 0$  then
    |  $[di, dj, nmax3] = \text{nextnecklace}(di, dj, nmax3, nmax, n)$ ;
  | end
  |  $[di, dj, nmax3] = \text{nextnecklace}(di, dj, nmax3, nmax, n)$ ;
end
for  $c$  from  $nc - des + 1$  to  $nc$  do
  |  $k = \text{mod}(k + q, n) + 1$ ;  $j = \text{mod}(k - des - 1, n) + 1$ ;
  |  $i = \text{mod}(j - des - 1, n) + 1$ ;
  | next kernel is “[i, j, k]”;
end

```

C.1 Auxiliary functions for Enhanced Shifting Necklaces (triads)

Algorithm 8: Auxiliary functions for Enhanced Shifting Necklaces

```

function  $[q, r] = \text{swap}(s, t)$ 
   $q = t;$ 
   $r = s;$ 
end

function  $[di, dj, nmax3] = \text{nextnecklace}(di, dj, nmax3, nmax, n)$ 
   $[di, dj] = \text{swap}(di, dj);$ 
   $maxin = \max(di, dj);$ 
   $minin = \min(di, dj);$ 
  if  $di \leq dj$  or  $2maxin == n - minin$  then
     $di = minin;$ 
     $dj = maxin;$ 
    if  $dj == nmax3$  then
      if  $di == nmax$  then
         $di = 1;$ 
         $dj = di;$ 
      else
         $di = di + 1;$ 
         $dj = di;$ 
      end
       $nmax3 = \text{floor}((n - di)/2);$ 
    else
       $dj = dj + 1;$ 
    end
  end
end
end

```
